

ExoHabitAI Deployment Documentation

Project Overview

Project Name: ExoHabitAI
Backend Framework: Flask
Frontend: HTML, CSS, JavaScript
Hosting Platform: Render
Version Control: GitHub

Project Architecture

User Browser → Frontend → Render Cloud Flask API → Machine Learning Model (.pkl)

Repository Structure

ExoHabitAI/
 app.py
 utils.py
 requirements.txt
 runtime.txt
 Procfile
 models/logistic_regression_final.pkl
 data/processed/
 frontend/index.html, script.js, style.css, bg.js

Deployment Platform

The project is deployed using Render, a cloud Platform-as-a-Service. Render provides automatic deployment from GitHub, dependency installation, server hosting, and environment management.

Deployment Steps

1. Upload all project files to GitHub repository.
2. Create a new Web Service on Render.
3. Connect GitHub repository and select main branch.
4. Set runtime to Python 3.11.9 using runtime.txt.
5. Add Procfile with Gunicorn start command.
6. Add dependencies in requirements.txt.

7. Deploy and wait for build to complete.
8. Access the live application URL.

Backend API Endpoints

Root Endpoint: GET / → Returns backend running message.

Health Endpoint: GET /health → Returns status OK.

Prediction Endpoint: POST /predict → Accepts JSON planet data and returns habitability prediction.

Model Integration

The machine learning model was trained using Scikit-learn and saved as a .pkl file using Joblib. The model is loaded once at application startup to improve performance.

Frontend Integration

The frontend form collects planet parameters and sends a POST request to the Render backend using Fetch API. The prediction result is displayed dynamically on the webpage.

Auto Deployment

Render automatically deploys the application whenever changes are pushed to GitHub. This enables Continuous Deployment and keeps the application updated.

Challenges Faced

During deployment, issues occurred such as Python version mismatch, dependency conflicts, missing files, and model compatibility problems. These were resolved by aligning package versions, correcting paths, and updating configuration files.

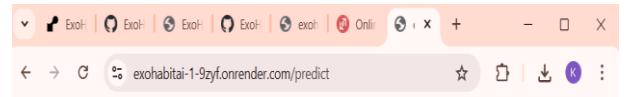
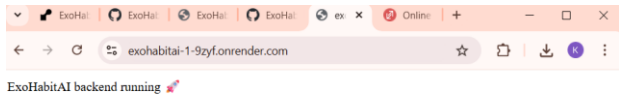
GitHub Repository URL

<https://github.com/BINGI-KEERTHANA/ExoHabitAI>

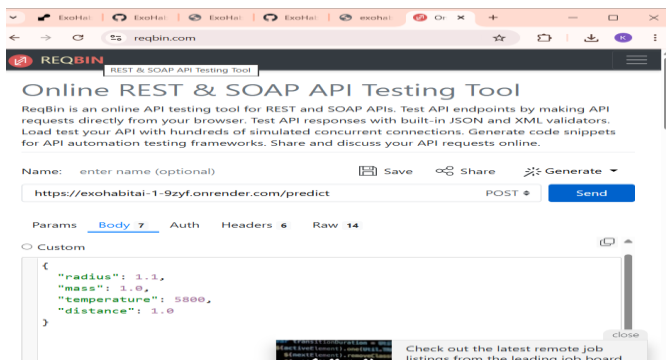
Final Deployment URL

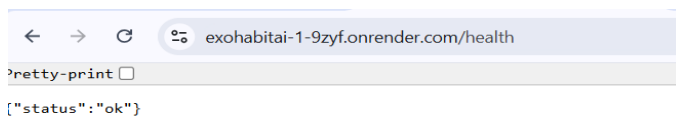
<https://exohabitai-1-9zyf.onrender.com>

Screenshot of working web app



Send POST request with JSON data





Conclusion

ExoHabitAI has been successfully deployed using Flask, Scikit-learn, GitHub, and Render cloud hosting. The system is fully functional, automatically deployable, and accessible online.